

ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY (AASTU)

COLLEGE OF ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING



INVENTORY MANAGEMENT SYSTEM

COURSE TITLE: Database Systems

COURSE CODE: SWEG2108

INSTRUCTOR'S NAME: Mr. Yaynshet

SECTION: A

GROUP MEMBERS	ID. NUMBER
1. Dagmawit Binyam	ETS 0400/17
2. Abel Yohannes	ETS 0039/17
3. Ashenafi G/Medhin	ETS 0202/17
4. Bereket Simegn	ETS 0267/17
5. Eden Alemayehu	ETS 0455/17
6. Edilawit Legesse	ETS 0462/17

SUBMITTED ON: MAY 22, 2026

Table of Contents

1: Introduction	4
1.1 Background of Organization	4
1.2 Data Processing Unit	5
1.3 Problem Statement	5
1.4 Objective	6
1.4.1 General Objective	6
1.4.2 Specific Objectives	6
1.5 SCOPE	6
1.6 Benefit	7
1.7 Methodology and Tools	7
2. Data base design	8
2.1 Entities and attributes	8
2.2 Relationship	10
2.3 Conceptual design (ERD)	10
2.4 Logical schema	11
2.5 Normalization	11
2.5.1 1NF (First Normal Form)	11
2.5.2 Second Normal Form (2NF)	12
2.5.3 Third Normal Form (3NF)	12
2.5.4 BCNF (Boyce-Codd Normal Form)	12
2.6 Inputs (forms), outputs, and reports	13
2.6.1 Inputs	13
2.6.2 Outputs	13
2.6.3 Report	14
3.Implementation	14
3.1 Introduction	14
3.2 MY SQL implementation	14
3.2.1 Database Creation	14
3.2.2 Table Implementation	14
3.2.3 Database Relationship Diagram	18
3.3 Sample Data Insertion	18
3.4 Query Execution / Reports	19

3.5 MongoDB Implementation.....	21
3.6 Testing.....	21
3.7 Results.....	22
3.8 Bugs Encountered.....	22
3.9 Improvements.....	23
Conclusion	24

ABSTRACT

In this project we develop a database system for managing rental properties by connecting property owners and users through a centralized platform. Currently, the process of finding rental houses is difficult due to inconsistent and scattered information, resulting in a time-consuming and inefficient experience. The proposed system addresses these challenges by allowing owners to register properties and create listings, while users can search and filter available listings based on their preferences. The system organizes property, listing, and search data to improve accessibility, transparency, and overall efficiency in the rental process. And this report shows the overall process of building the project.

1: Introduction

In our daily life, we rely on data to make decisions ranging from some pieces of information to large volumes of data. Database systems play an important role in organizing and accessing this information efficiently, providing many advantages to us over the old manual data handling methods.

1.1 Background of Organization

The real estate rental system

The real-estate rental sector is an essential part of urban and rural development. It facilitates the allocation of properties to individuals, families, and businesses. It has multiple components, including property owners who provide rental properties, tenants who will rent or buy the property, and brokers who act as intermediaries between both parties. In many developing countries including our country Ethiopia, this sector plays a crucial role in ensuring access to housing, especially in rapidly growing cities like Addis Ababa where demand for rental properties is high. Despite its importance, the system is still largely dependent on informal communication channels and traditional methods. These include physical visits, personal contacts, and broker-based interactions rather than structured digital platforms. As a result, the real estate rental sector lacks a unified and efficient system for managing property information and communication between owners and tenants.

1.2 Data Processing Unit

Existing System

In the current real estate rental environment, we have observed that data processing is mainly handled through informal and manual methods. Property owners typically depend on brokers or social media platforms such as Facebook groups and pages to advertise available houses. Brokers play a central role by collecting property information from owners and communicating it to tenants. In addition, many tenants rely on visiting different brokers physically or checking online posts to find available rental houses. This problem is common in our surroundings and families. And this process is often exhausting and time-consuming. People are required to move from one broker to another or search through multiple locations to confirm the property's availability and negotiate with price. This manual approach to data handling creates confusion, delays in decision-making, and unnecessary effort for both tenants and property owners.

1.3 Problem Statement

From the existing system's background, we can surely say that the real estate rental system lacks a centralized and efficient platform for doing all the things that are performed manually. As a result, property owners, brokers, and tenants rely on exhausting methods such as physical visits, broker mediation, and social media platforms to exchange rental information. This leads to several challenges, including

- Inconsistent and outdated property listings.
- Difficulty in verifying availability.
- Delays in communication between parties (tenants and owners).

Tenants often waste significant time searching for suitable houses, while owners struggle to reach genuine and interested clients efficiently. Additionally, the heavy dependence on brokers introduces issues related to transparency and standardization of rental information. Therefore, there is a clear need for a structured system that can provide the rental process, improve data accuracy, and enhance communication between all stakeholders involved in the real estate rental market.

1.4 Objective

1.4.1 General Objective

From the problem we observed, we aim to design a structured data base for real estate management system that simplifies and improves the data exchange, communication, and make information sharing more efficient between property owners, brokers, and tenants.

1.4.2 Specific Objectives

- To reduce the effort and time required for tenants to find available rental houses by providing a centralized system for property listings.
- To minimize the need for tenants to physically move from one broker to another or search for different locations when looking for housing information.
- To reduce dependency on traditional broker-based searching by enabling direct access to property information through the system.
- To improve communication between property owners, brokers, and tenants to make the rental process more efficient and less exhausting.
- To provide updated and reliable property information.
- To reduce confusion caused by inconsistent or repeated listings.

1.5 SCOPE

The scope of this project is to develop a centralized real estate rental listing system that provides a platform for property owners to publish rental properties and for potential tenants to search and view available houses.

The system offers functionality for:

- Property listing: where owners of the property can input details such as location, price, description, and availability.
- Search and filtering feature: where users browse properties based on their preferences.
- Contact information of property owners: display the information to enable direct communication between tenants and owners.

However, the system does not include:

- Financial transaction processing, such as online payments or money transfers.
- Legal agreements.
- Contracts, or property ownership registration.

- Physical verification of properties or government-level property management.

The system is strictly limited to three basic things: managing rental listings, enabling search functionality, and facilitating communication between users.

1.6 Benefit

Our proposed real estate rental system provides several advantages compared to the traditional broker that requires manual data handling and many physical contacts.

One of the major benefits of the system is the reduction of time and effort required to find rental houses. Instead of moving from one broker to another or visiting different locations, tenants can search for available properties directly through the system. This significantly reduces physical movement and improves efficiency in the rental process.

Another advantage is improved communication between property owners and tenants. The system allows direct access to owner contact information, eliminating unnecessary intermediaries and reducing misunderstandings caused by broker communication. Property owners also benefit by reaching a wider audience without depending on physical brokers or informal advertising methods.

Simply, it can surely be said that our proposed system has a win-win solution for both the owners and the tenants.

1.7 Methodology and Tools

While developing this project, we followed the theoretical knowledge we got from data base class that includes requirement gathering (planning), system analysis, system design, implementation, and some testing. In the requirement gathering phase, we collected information about the current real estate rental process through direct observation and informal discussions with tenants, property owners, and brokers that we found in our environment. This helps us in understanding the main problems in the existing system. In the system analysis and design phase, we analyzed the collected requirements to define the system structure, including database design and system flow. In this phase we designed the ER diagram and schema for better understanding of the system. After that, our implementation phase involves developing the system based on the designed models. Finally, we tested the system to ensure that it functions correctly and meets the intended requirements.

While all this methodology and phase we used various tools. For the ER diagram we used LUCID Charts, for the code we use MYSQL and MONGODB, for documentation we used MICROSOFT WORD. The introduction to data base theory helped us to do observation, problem identification, and do background research on the organization we chose.

2. Data base design

Database design is a systematic process that involves a set of procedures and tasks to implement a database effectively. It focuses on organizing and structuring data in a way that ensures efficiency, accuracy, and usability. In order to build the data base system that we assume as a solution for the current system, we were supposed to follow this stage properly. The data base design phase consists logical design, physical design and normalization.

Logical Data Model Design: In this stage a high level structure of the data base is created. It identifies entities, attributes, keys, relationships, and constraints without considering physical storage. The ER modeling and normalization are also used in this stage to organize data and remove redundancy.

Physical Data Model Design: In this stage the logical design is implemented in DBMS. Tables are created, relationships and constraints are defined, and storage details are set to generate the actual working database.

Normalization : Systematic process of organizing data into smaller tables to reduce redundancy and dependency.

So for building the right database design we have tried to use ER diagram for logical design, the schema for the physical data model, and normalization for solving data redundancy.

2.1 Entities and attributes

For a recall, entities are the real world objects that have properties named attributes. So for the real estate rental system we will list out the objects and the properties of each object to define the relationship between them.

Entities with their corresponding attributes

1. USER

The user entity stores information about all users of the system, including both roles (property owners and tenants). We merge the role of owners and tenant as user to reduce data redundancy because both have the same attributes.

Attributes

- user_id (PK) – unique identifier for each user, and it is the primary key.
- full_name – name of the user.
- email – user's email address .
- phone – contact number .
- password – used for authentication .
- role – defines whether the user is an owner or tenant.

- created_at – date the account was created.

2. PROPERTY

The property entity represents rental listings created by property owners. We build the system with allowance to owners to list property so the attributes of each property is listed by the owner.

Attributes

- property_id (PK) – unique identifier for each property, the primary key for the property.
- title – short title of the listing, used for catching users attention.
- description – detailed information about the property.
- property_type – type (house, apartment, etc.) .
- price – rental price .
- bedrooms – number of bedrooms .
- area_size – size of the property (e.g.care) .
- status – availability (*available, rented, not available*) .

3. LOCATION

The location entity stores general location information for properties. We make it separated to avoid repeating city/ sub city for every property.

Attributes

- location_id (PK) – unique identifier, it is the primary key.
- city – city where the property is located.
- subcity – sub-area within the city.

4. Property photo

The properties photo entity stores images related to properties. This helps the tenants to see the property without being physically there. Property does not store images directly to avoid duplication because one owner may enter 3 photos, other may have single photo so for a better structure we make it a separate entity.

Attributes

- photo_id (PK) – unique identifier, it is the primary key.
- photo_url – path or link to the image.

2.2 Relationship

A relationship defines how entities are logically connected. It can be unary, binary and ternary.

1. USER — LISTS — PROPERTY

$1 \rightarrow N$

One user can list many properties.

Each property belongs to one user.

2. PROPERTY — LOCATED IN — LOCATION

$N \rightarrow 1$

Many properties can exist in the same area.

Each property has only one location.

3. PROPERTY — HAS PHOTO — PROPERTY_PHOTO

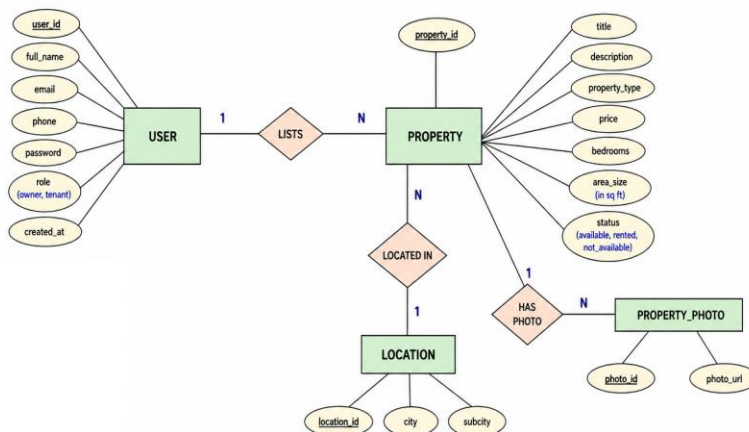
$1 \rightarrow N$

A property can have multiple photos.

Each photo belongs to one property.

2.3 Conceptual design (ERD)

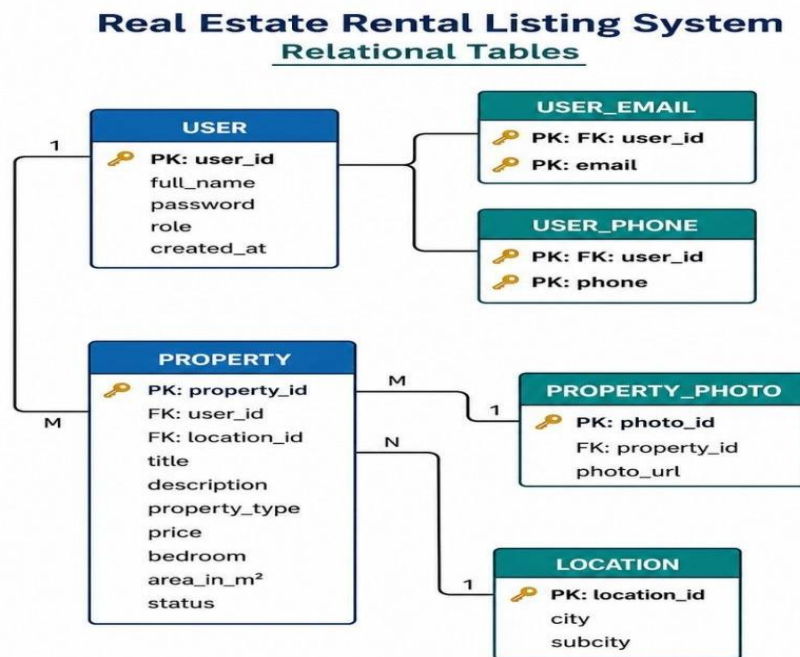
So, from the above entities and attributes we have build a relationship right? From all that basic information we built the entity relation diagram.



ER DIAGRAM – REAL ESTATE RENTAL LISTING SYSTEM

2.4 Logical schema

Now from the ER we have a clear image. So we convert the ER into a relational logical design, the resulting relationship diagram is as follows:



The main purpose of the physical design is to actually implement the logical design that is, showing the structure of the database along with all the columns & their data types, rows, relations, relationships among data & clearly define how relations are related to each other.

2.5 Normalization

It was an important process in our database design. It helps us to improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs in the real estate business.

In order to achieve this stage we studied deeply about normalization and its types upto BCNF and apply them on our design.

2.5.1 1NF (First Normal Form)

Rules for 1NF are easy.” All attributes are single valued!”.

In order to achieve this rule we must do two things. First identify if there is any problem. In our table phone has multiple values and email also has multiple values. We did this to make it efficient for both tenants and owners with sustainable contact information. Second fix the problem. We achieve that by splitting the multivalued attributes.

The result becomes:

USER(user_id(PK), full name, password, name)

USER_PHONE(phone_id (PK), user_id (FK) , phone)

USER_EMAIL(email_id (PK), user_id (FK), email)

Now 1NF is done successfully!!!

2.5.2 Second Normal Form (2NF)

Rule for 2NF is “Should be on 1NF and no partial dependency”.

1NF was done before, and for the second rule even on composite ID like user, user phone and user email we don't have partial dependency we have full dependency. Now we can surely say that 2NF is done successfully and proceed.

2.5.3 Third Normal Form (3NF)

The rule for 3NF is “Should be on 2NF and no transitive property”.

In our design both are satisfied.

- USER → depend on user_id
- PROPERTY → depends on property_id
- LOCATION → separated from user
- USER_PHONE → phone depends only on phone_id

2.5.4 BCNF (Boyce-Codd Normal Form)

Rule for BCNF is “Satisfy 3NF and Every determinant must be a candidate key”.

In our system, the design satisfies BCNF as all determinants are candidate keys and there are no functional dependencies violating BCNF conditions.

NORMALIZATION SUMMARY TABLE

Normal Form	Rule	Issue in Initial Design (A)	Solution Applied	Result
1NF (First Normal Form)	All attributes must be atomic. No repeating groups or multivalued attributes.	- phone stored as multiple values (e.g., "0911, 0922") in one field. - email stored as multiple values (e.g., "a@mail.com, b@mail.com") in one field.	Decomposed multivalued attributes into separate tables: USER_PHONE (phone_id, user_id, phone) USER_EMAIL (email_id, user_id, email)	All attributes are atomic. Each table contains single values only. Satisfies 1NF.
2NF (Second Normal Form)	Must be in 1NF. No partial dependency of any non-key attribute on a part of a composite key.	No issue. All tables have single attribute primary keys (user_id, phone_id, email_id, property_id, location_id, etc.).	No change needed.	No partial dependencies exist. Satisfies 2NF.
3NF (Third Normal Form)	Must be in 2NF. No transitive dependency (non-key attribute should not depend on another non-key attribute).	In PROPERTY table (if location was not separated): - city → subcity - property_id → city → subcity This is a transitive dependency.	Created separate LOCATION table: LOCATION (location_id, city, subcity) PROPERTY now stores only location_id (FK).	Transitive dependency removed. All non-key attributes depend only on the primary key. Satisfies 3NF.
BCNF (Boyce-Codd Normal Form)	For every functional dependency $X \rightarrow Y$, X must be a candidate key (super key).	No determinant violations observed in the decomposed tables.	All tables designed so that every determinant is a candidate key.	All determinants are candidate keys. Satisfies BCNF.
Conclusion The initial design contained multivalued attributes and potential transitive dependencies. Through normalization, the schema was decomposed into multiple relations to eliminate redundancy and ensure data integrity. The final design satisfies all normalization requirements up to BCNF.				

In general now shout out to our design, it is normalized to BCNF. Multivalued attributes such as phone numbers and email addresses have been separated into independent relations to satisfy (1NF). All relations have single primary keys, eliminating partial dependencies (2NF), and transitive dependencies have been removed (3NF). Furthermore, all determinants are candidate keys, ensuring compliance with BCNF.

2.6 Inputs (forms), outputs, and reports

2.6.1 Inputs

These are the forms users interact with.

Basically, from the design we will have some information to accept.

(i) User Registration Form

In this form we expect to accept inputs from the users both tenant and owner. It includes: full_name, email, phone, password, role(owner,tenant).

(ii) Login Form

This form is responsible for login purposes. If the user is already registered to the database, there is no need of reregistration. So, in this form the inputs will be passwords and emails.

(iii) Add Property Form (Owner only)

This form is only required from the owner, because listing the house is the responsibility of owner. The fields over this form are: title, description, property_type, price, bedrooms, area_size, location (city, subcity), status, upload photo(s)

(iv) Upload Photo Form

This form is to attach a picture of the property, so, the field is the photo_file.

2.6.2 Outputs

This design shows what the system displays to the user.

i) Property Listing Page

It shows multiple properties of the property inputted by the owner.

The displayed data: Title, price, location, bedrooms, images.

ii) **Property Detail Page**

Shows full information of the property. The displayed data are all the inputs by the owner including contact of the user.

2.6.3 Report

Reports are more organized or summarized outputs.

(i) **Property Report:** It lists all the properties in the system.

(ii) **Available Properties Report:** show only available houses.

So, by using our unit 2's theoretical knowledge, we design the database system, including identifying entities and relationships, creating an ER diagram, converting it into a logical schema, and applying normalization up to BCNF. We also include the design of inputs, outputs, and reports to ensure efficient data entry, processing, and presentation.

3.Implementation

3.1 Introduction

In the implementation phase, we used MySQL and MongoDB to manage and store our rental data. During the design phase, we noticed that the system contains multiple related entities, relationships, and structured data. Therefore, we selected MySQL because it efficiently handles relational data, supports structured queries, and maintains relationships using primary and foreign keys.

In addition, MongoDB was used to demonstrate the NoSQL implementation of the system using collections and documents. Its flexible document-based structure allows related information to be stored efficiently and supports scalable data management.

The implementation includes creating tables and collections, defining relationships using primary and foreign keys in MySQL, inserting sample data, and enforcing data integrity constraints to ensure consistency and reliability within the system.

3.2 MY SQL implementation

3.2.1 Database Creation

First a database named "Rreal_estate_rental" was created to store and manage all data related to the rental system.

3.2.2 Table Implementation

After creating the data base name various tables were directly created from the logical schema. Each attribute in the table is stored using an appropriate data type to ensure accurate and efficient data storage.

- User table

The “user” table was created to store information about all users in the system, it includes both tenants and property owners.

The table contains each details of users including : ‘Name’, ‘password’, ‘role’, and ‘account creation date’. The role attribute determines whether the user is tenant or owner. And the created_at automatically the date and time the user account is created at.

Its primary key is the user_id.

```
CREATE TABLE user (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    full_name VARCHAR(100) NOT NULL,  
    password VARCHAR(100) NOT NULL,  
    role ENUM('owner', 'tenant') NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Figure 3.1: Structure of the user table.

- User email table

The “user email “ table stores email addresses of users. It has one to many relationship, which allows one user to have multiple emails. That is the main reason it becomes a table. Because if we only allow the user to have one email for the system the email will have been an attribute to the user table .But, for safe contact information for tenants and also owners of properties we make the relation one user to many emails.

The table uses email_id as its primary key and user_id as a foreign key to connect each email address to its user.

```
create table user_email (  
    email_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES user(user_id)  
        ON DELETE CASCADE  
);
```

Figure 3.2: Structure of the user_email table.

- User phone table

The user phone table stores the phone number associated with the user in the system. Similar to the previous user_email table, it follows a one-to-many relationship.

We also separated this information as its own table to provide flexible and reliable communication methods for tenants and property owners. If the system only allowed one phone number per user, it could have been stored directly in the user table. However, the separate table design allows the system to manage multiple contact numbers for a single user.

The table uses phone_id as the primary key and user_id as a foreign key to connect phone numbers with their users.

```
create table user_phone (  
    phone_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT,  
    phone VARCHAR(20) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES user(user_id)  
        ON DELETE CASCADE  
);
```

Figure 3.3: Structure of the user_phone table.

- Location table

The location table stores the geographical location of the properties. It contains details such as city and subcity.

In our environment there are many properties that are located on same spot. so multiple properties might have the same location. Separating location information into its own table reduces data redundancy and improves database organization.

The table uses location_id as the primary key to uniquely identify each location record.

```
CREATE TABLE location (  
    location_id INT AUTO_INCREMENT PRIMARY KEY,  
    city VARCHAR(50) NOT NULL,  
    subcity VARCHAR(50) NOT NULL  
);
```

Figure 3.4: Structure of the

location table.

- Property Table

The property table was created to store detailed information about rental properties listed in the system. It is the main table of the database because it contains the core information we want for managing property rentals. The table has details such as title, description, type, price, number of bedrooms, area size, and property status. The status attribute is so crucial for the tenants by indicating whether a property is available, rented, or not available.

The table also contains foreign keys that create relationships with the user and location tables. The user_id foreign key connects each property to its owner, while the location_id foreign key links the property to its location.

The table uses property_id as the primary key to uniquely identify each property record.

```
create table property (  
  property_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT,  
  location_id INT,  
  title VARCHAR(100),  
  description TEXT,  
  property_type VARCHAR(50),  
  price DECIMAL(10,2),  
  bedrooms INT,  
  area_size DECIMAL(10,2),  
  status ENUM('available','rented','not available') DEFAULT 'available',  
  FOREIGN KEY (user_id) REFERENCES user(user_id)  
    ON DELETE CASCADE,  
  FOREIGN KEY (location_id) REFERENCES location(location_id)  
    ON DELETE CASCADE  
);
```

Figure 3.5: Structure of the property table.

- Property Photo Table

The property_photo table was created to store images of the properties in the system. It follows a one-to-many relationship. This allows one property to have multiple photos to ensure that the tenants have enough image to look in the system instead of physically going there.

The table stores the photo URL for each image and connects it to the property using a foreign key relationship. The table uses photo_id as the primary key and property_id as a foreign key to associate each photo with its corresponding property.

```
create table property_photo (  
  photo_id INT AUTO_INCREMENT PRIMARY KEY,  
  property_id INT,  
  photo_url VARCHAR(255),  
  
  FOREIGN KEY (property_id) REFERENCES property(property_id)  
    ON DELETE CASCADE  
);
```

Figure 3.6: Structure of the property_photo table.

3.2.3 Database Relationship Diagram

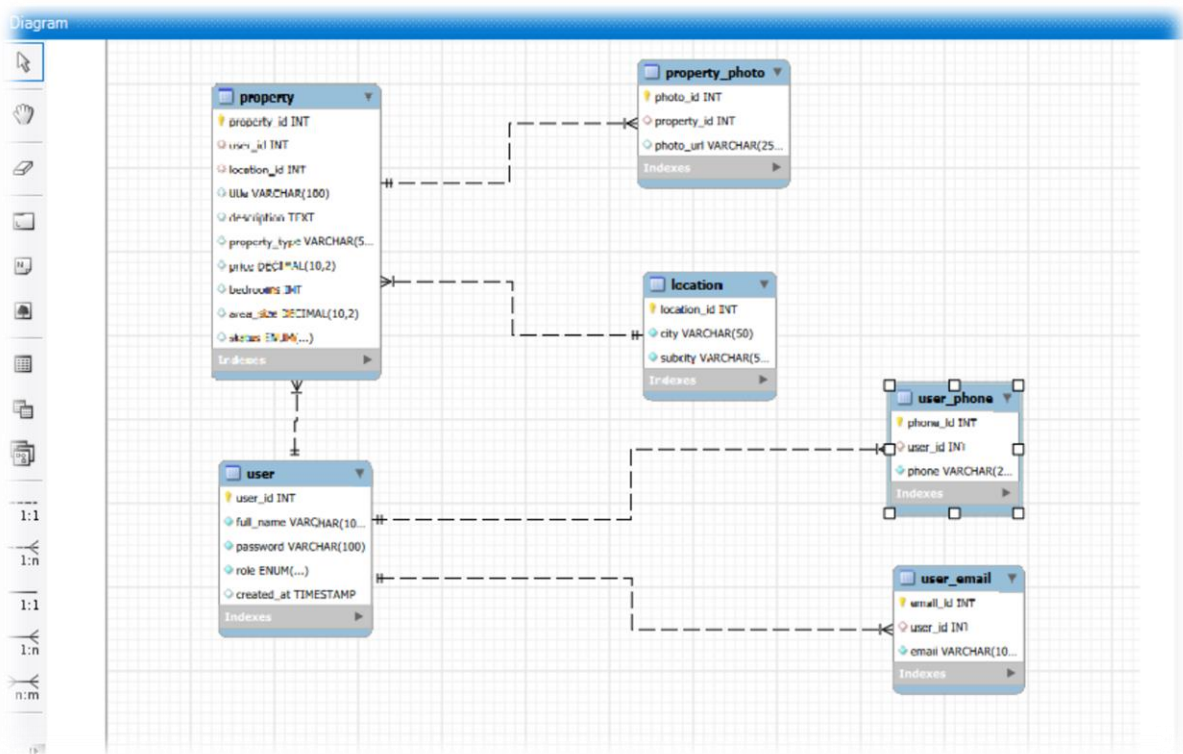


Figure 3.7: EER diagram showing relationships between the implemented database tables.

The figure above clearly shows the relationships between the tables in the implemented database. The relationships were created using primary keys and foreign keys to ensure data integrity and proper connectivity between entities. And we get this enhanced entity diagram from the MYSQL workbench using the reverse engineer approach is different from the ER diagram it is implementation from the actual database relationship diagram and is generated from real tables. It shows the clean relationship between each entities.

3.3 Sample Data Insertion

Sample data was inserted into the database tables to test the functionality of the system and verify that relationships between entities were correctly implemented. So, for testing the functionality of our data base sysytem we have added sample data. Before inserting sample records, we cleared existing data in the tables using the TRUNCATE command. Foreign key checks were also temporarily disabled to avoid relational conflicts during the deletion process.

```

SET FOREIGN_KEY_CHECKS = 0;

TRUNCATE TABLE property_photo;
TRUNCATE TABLE property;
TRUNCATE TABLE user_email;
TRUNCATE TABLE user_phone;
TRUNCATE TABLE user;
TRUNCATE TABLE location;

SET FOREIGN_KEY_CHECKS = 1;

```

Figure 3.8: TRUNCATE on the sample data.

Property and Location Data

```

INSERT INTO property (user_id, location_id, title, description, property_type, price, bedrooms, area_size, status)
SELECT user_id, 1, 'Modern Apartment', 'Nice apartment in Sole', 'Apartment', 15000, 3, 120, 'available'
FROM user WHERE full_name='Abel Yohannis';

```

This screenshot is one of the sample data inserted property ,it was linked with locations and property images using foreign key relationships.

3.4 Query Execution / Reports

SQL queries were executed to test data retrieval and verify that the implemented database functions correctly. The queries were used to retrieve property information, user details, and relationships between entities.

The following query retrieves all property records stored in the system.

```

use Rreal_estate_rental;
SELECT * FROM property;

```

The query below retrieves only properties that are currently available for rent.

```

use Rreal_estate_rental;
SELECT *
FROM property
WHERE status = 'available';

```

The following query retrieves property information together with location details and property images by joining the property, location, and property_photo tables.

```
SELECT p.title, p.price, p.bedrooms, l.city, l.subcity, ph.photo_url
FROM property p
JOIN location l ON p.location_id = l.location_id
LEFT JOIN property_photo ph ON p.property_id = ph.property_id;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	title	price	bedrooms	city	subcity	photo_url
▶	Modern Apartment	15000.00	3	Addis Ababa	Bole	https://homebnc.com/homeimg/2020/0
	Modern Apartment	15000.00	3	Addis Ababa	Bole	https://cloudfrontgharpediabucket.gha
	Family House	20000.00	4	Addis Ababa	Kirkos	https://cdn.decorilla.com/online-decora
	Studio Room	8000.00	1	Addis Ababa	Yeka	https://tse2.mm.bing.net/th/id/OIP.us

The following query retrieves complete information about a specific property, including owner details, contact information, and location data.

```
SELECT p.*, u.full_name, ue.email, up.phone, l.city, l.subcity
FROM property p
JOIN user u ON p.user_id = u.user_id
JOIN user_email ue ON u.user_id = ue.user_id
JOIN user_phone up ON u.user_id = up.user_id
JOIN location l ON p.location_id = l.location_id
WHERE p.property_id = 1;
```

property_id	user_id	location_id	title	description	property_type	price	bedrooms	area_size	status	full_name	email	phone	city	subcity
1	1	1	Modern Apartment	Nice apartment in Bole	Apartment	15000.00	3	120.00	available	Abel Yohannis	abel@gmail.com	0911451811	Addis Ababa	Bole

The following query retrieves available properties based on price range and minimum number of bedrooms.

```
SELECT *
FROM property
WHERE price <= 20000
AND bedrooms >= 2
AND status = 'available';
```

property_id	user_id	location_id	title	description	property_type	price	bedrooms	area_size	status
1	1	1	Modern Apartment	Nice apartment in Bole	Apartment	15000.00	3	120.00	available
2	3	2	Family House	Spacious house in Kirkos	House	20000.00	4	200.00	available
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

The executed SQL queries successfully retrieved and filtered data from multiple related tables in the database. The results confirmed that the relationships between entities were properly implemented and that the system can efficiently manage property, user, location, and contact information. The queries also demonstrated accurate data retrieval, filtering, and reporting functionalities within the real estate rental system.

3.5 MongoDB Implementation

In addition to MySQL, we used MongoDB to implement the NoSQL part of the system. MongoDB stores data in collections and documents using a flexible JSON-like structure. This approach allows related data to be stored together in a single document.

The MongoDB implementation successfully stored rental information using document-based collections. Queries executed on the collections returned accurate results and demonstrated flexible data retrieval capabilities.

3.6 Testing

We performed testing to verify that the database functions correctly and that relationships between tables operate as expected. Different operations such as data insertion, retrieval, filtering, and relationship validation were tested during the implementation process.

Test Case	Operation Performed	Expected Result	Actual Result	Status
User insertion	Insert user record	User stored successfully	User inserted successfully	Passed
Property insertion	Insert property data	Property linked to owner and location	Property stored correctly	Passed
Email relationship	Insert multiple emails for one user	Multiple emails linked to same user	Relationship worked correctly	Passed
Query execution	Retrieve available properties	Only available properties displayed	Correct results returned	Passed
Property photos	Insert multiple photos for one property	Photos linked correctly	Relationship worked correctly	Passed

Through this process we confirmed that the database implementation functions correctly. The tables, relationships, and queries operated as expected, and the system successfully managed and retrieved rental data without major issues.

3.7 Results

The implementation phase of the real estate rental system was completed successfully using MySQL and MongoDB as database management systems. The database structure we developed was able to store, organize, and manage rental information efficiently while maintaining proper relationships between entities.

In the MySQL implementation, we created tables successfully with their corresponding primary keys and foreign key constraints. The relationships between users, properties, locations, contact information, and property photos functioned correctly and maintained data integrity throughout the implementation process. And we use relational constraints to ensure consistency and reduced redundancy within the database.

After the creation of tables and data base we inserted sample data into all tables successfully, including users, email addresses, phone numbers, locations, properties, and property images. The inserted data verified that the database structure supports one-to-many relationships such as one user having multiple phone numbers or email addresses, and one property containing multiple photos.

Different SQL queries were executed to test data retrieval, filtering, and reporting operations. The executed queries successfully retrieved detailed property information, owner contact details, location information, and property images from multiple related tables. Filtering queries also returned accurate results based on conditions such as property price, number of bedrooms, and availability status.

The MongoDB implementation also demonstrated successful storage and retrieval of rental data using collections and documents. The document-based structure provided flexibility in storing related information together within single documents. Queries executed on the collections returned the expected results and confirmed that the NoSQL implementation functions properly.

The testing process confirmed that the implemented database systems are capable of managing rental data accurately and efficiently. Operations such as inserting records, retrieving data, and maintaining relationships between entities were performed successfully without major issues. Overall, the implementation achieved the intended objectives of the project and provided a functional database system for managing real estate rental information.

3.8 Bugs Encountered

During the implementation process, several challenges and minor issues were encountered while developing and testing the database system. Most of these issues were related to foreign key relationships, query execution, and data insertion operations. One of the common problems encountered was foreign key constraint errors when inserting or deleting related

records. This issue occurred because some tables depended on records from other tables. The problem was resolved by inserting records in the correct order and temporarily disabling foreign key checks when clearing tables for testing purposes.

Another issue involved duplicate data insertion, especially when testing email addresses and phone numbers. Since the `user_email` table used a unique constraint for email addresses, duplicate email entries generated errors. This helped ensure data consistency and prevented repeated records in the database.

In addition, minor syntax and execution errors occurred during the implementation process, including incorrect table references and missing values in insertion statements. These problems were identified and corrected through testing and debugging.

Despite these challenges, we resolved the identified bugs resolved successfully as a team, and the database system functioned correctly after all the effort we made together.

3.9 Improvements

Although the implemented real estate rental system functions successfully, several improvements can be made in the future to enhance the system's performance, usability, and security.

One possible improvement is the development of a graphical user interface or web-based application that allows users to interact with the system more easily. This would improve user experience by enabling property owners and tenants to manage rental information through a user-friendly environment instead of directly interacting with the database.

Another improvement would be adding advanced property search and filtering features. Users could search properties based on additional criteria such as location, price range, property type, number of bedrooms, and availability status. This would make the system more efficient and convenient for tenants searching for rental properties.

Security can also be improved by implementing password encryption and stronger authentication mechanisms. Currently, passwords are stored directly in the database for testing purposes, but future versions of the system should use secure password hashing techniques to protect user information.

Additional features such as online property image uploads, rental payment management, booking functionality, and notification systems could also be integrated into the system. These features would expand the functionality of the project and make it more suitable for real-world use.

Furthermore, the database can be optimized to improve performance when handling large amounts of rental data. Indexing, query optimization, and better database management techniques could help increase the efficiency and scalability of the system. Overall, these

improvements would enhance the functionality, security, and usability of the real estate rental system and support future expansion of the project.

Conclusion

This project provided a valuable learning experience throughout the research, design, implementation, testing, and debugging phases of database system development. The main objective of the project was to solve the problem of inefficient management of rental property information. Traditional methods of managing rental data are often unorganized, time-consuming, and difficult to maintain, especially when handling large amounts of property, tenant, and owner information. Therefore, the system was designed to provide a structured and efficient way of storing, managing, and retrieving real estate rental data.

Beyond the technical knowledge gained, the project also demonstrated the importance of teamwork, communication, and collaboration. Working together as database students allowed the team to exchange ideas, solve problems collectively, and support one another throughout the development process. Each phase of the project became part of a learning journey that improved both technical understanding and cooperative working skills.

Overall, the project successfully achieved its objectives and provided practical experience in designing and implementing database systems. It also created a strong foundation for future learning and development in the field of database management and software systems.

References

- [1] Zillow, “Real Estate and Rental Listings.” Available: <https://www.zillow.com/>

- [2] Airbnb, “Property Rental Platform.” Available: <https://www.airbnb.com/>

- [3] Apartments.com, “Apartment and Property Rental Listings.” Available: <https://www.apartments.com/>

- [4] R. Elmasri and S. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

- [5] GeeksforGeeks, “DBMS Tutorials.” Available: <https://www.geeksforgeeks.org/dbms/>

- [6] Oracle Corporation, “MySQL Documentation.” Available: <https://dev.mysql.com/doc/>

- [7] MongoDB Inc., “MongoDB Manual.” Available: <https://www.mongodb.com/docs/>

ANNEX

A.MY SQL CODE

TABLE AND DATA BASE CREATION

```
CREATE DATABASE Rreal_estate_rental;
```

```
USE Rreal_estate_rental;
```

```
CREATE TABLE user (  
    user_id INT AUTO_INCREMENT PRIMARY  
KEY,
```

```
    full_name VARCHAR(100) NOT NULL,
```

```
    password VARCHAR(100) NOT NULL,
```

```
    role ENUM('owner','tenant') NOT NULL,
```

```
    created_at TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP
```

```
);
```

```
create table user_email (  
    email_id INT AUTO_INCREMENT PRIMARY
```

```
KEY,
```

```
    user_id INT,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,
```

```
    FOREIGN KEY (user_id) REFERENCES  
user(user_id)
```

```
    ON DELETE CASCADE
```

```
);
```

```
create table user_phone (  
    phone_id INT AUTO_INCREMENT PRIMARY
```

```
KEY,
```

```
    user_id INT,
```

```
    phone VARCHAR(20) NOT NULL,
```

```
    FOREIGN KEY (user_id) REFERENCES  
user(user_id)
```

```
    ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE location (  
    location_id INT AUTO_INCREMENT
```

```
PRIMARY KEY,
```

```
    city VARCHAR(50) NOT NULL,
```

```
    subcity VARCHAR(50) NOT NULL
```

```
);
```

```
create table property (  
    property_id INT AUTO_INCREMENT
```

```
PRIMARY KEY,
```

```
    user_id INT,
```

```
    location_id INT,
```

```
    title VARCHAR(100),
```

```
    description TEXT,
```

```
    property_type VARCHAR(50),
```

```
    price DECIMAL(10,2),
```

```
    bedrooms INT,
```

```
    area_size DECIMAL(10,2),
```

```
    status ENUM('available','rented','not  
available') DEFAULT 'available',
```

```
    FOREIGN KEY (user_id) REFERENCES  
user(user_id)
```

```
    ON DELETE CASCADE,
```

```
    FOREIGN KEY (location_id) REFERENCES  
location(location_id)
```

```
    ON DELETE CASCADE
```

```
);
```

```
create table property_photo (  
    photo_id INT AUTO_INCREMENT PRIMARY
```

```
KEY,
```

```
    property_id INT,
```

```
    photo_url VARCHAR(255),
```

```
    FOREIGN KEY (property_id) REFERENCES  
property(property_id)
```

```
    ON DELETE CASCADE
```

```
);
```

```
SAMPLE DATA
```

```
USE Rreal_estate_rental;
```

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
TRUNCATE TABLE property_photo;
```

```
TRUNCATE TABLE property;
```

```
TRUNCATE TABLE user_email;
```

```
TRUNCATE TABLE user_phone;
```

```
TRUNCATE TABLE user;
```

```
TRUNCATE TABLE location;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

```
INSERT INTO user (full_name, password, role)
VALUES
```

```
('Abel Yohannis', 'abel1323', 'owner'),
```

```
('Bereket Simegn', 'bereket8823', 'tenant'),
```

```
('Dagmawit Binyam', 'dagma124', 'owner'),
```

```
('Ashenafi Gebremedhin', 'ashenafi1243',
'tenant'),
```

```
('Eden Alemayehu', 'eden1123', 'owner');
```

```
INSERT INTO user_email (user_id, email)
```

```
SELECT user_id, 'abel@gmail.com' FROM user
WHERE full_name='Abel Yohannis';
```

```
INSERT INTO user_email (user_id, email)
```

```
SELECT user_id, 'bereket@gmail.com' FROM
user WHERE full_name='Bereket Simegn';
```

```
INSERT INTO user_email (user_id, email)
```

```
SELECT user_id, 'dagmawit@gmail.com' FROM
user WHERE full_name='Dagmawit Binyam';
```

```
INSERT INTO user_email (user_id, email)
```

```
SELECT user_id, 'ashenafi@gmail.com' FROM
user WHERE full_name='Ashenafi
Gebremedhin';
```

```
INSERT INTO user_email (user_id, email)
```

```
SELECT user_id, 'eden@gmail.com' FROM
user WHERE full_name='Eden Alemayehu';
```

```
INSERT INTO user_phone (user_id, phone)
```

```
SELECT user_id, '0911451811' FROM user
WHERE full_name='Abel Yohannis';
```

```
INSERT INTO user_phone (user_id, phone)
```

```
SELECT user_id, '0922672222' FROM user
WHERE full_name='Bereket Simegn';
```

```
INSERT INTO user_phone (user_id, phone)
```

```
SELECT user_id, '0933333333' FROM user
WHERE full_name='Dagmawit Binyam';
```

```
INSERT INTO user_phone (user_id, phone)
```

```
SELECT user_id, '0948944444' FROM user
WHERE full_name='Ashenafi Gebremedhin';
```

```
INSERT INTO user_phone (user_id, phone)
```

```
SELECT user_id, '0955554795' FROM user
WHERE full_name='Eden Alemayehu';
```

```
INSERT INTO location (city, subcity) VALUES
```

```
('Addis Ababa', 'Bole'),
```

```
('Addis Ababa', 'Kirkos'),
```

```
('Addis Ababa', 'Yeka');
```

```
-- INSERT PROPERTIES (ONLY OWNERS)
```

```
INSERT INTO property (user_id, location_id,
title, description, property_type, price,
bedrooms, area_size, status)
```

```
SELECT user_id, 1, 'Modern Apartment', 'Nice
apartment in Bole', 'Apartment', 15000, 3, 120,
'available'
```

```
FROM user WHERE full_name='Abel Yohannis';
```

```
INSERT INTO property (user_id, location_id,
title, description, property_type, price,
bedrooms, area_size, status)
```

```
SELECT user_id, 2, 'Family House', 'Spacious
house in Kirkos', 'House', 20000, 4, 200,
'available'
```

```
FROM user WHERE full_name='Dagmawit
Binyam';
```

```
INSERT INTO property (user_id, location_id,
title, description, property_type, price,
bedrooms, area_size, status)
```

```
SELECT user_id, 3, 'Studio Room', 'Affordable
studio in Yeka', 'Studio', 8000, 1, 60, 'available'
```

```
FROM user WHERE full_name='Eden
Alemayehu';
```

```
-- INSERT PHOTO
```

```
INSERT INTO property_photo (property_id,
photo_url)
```

```
SELECT property_id,
'https://homebnc.com/homeimg/2020/09/03e-
small-apartment-living-room-decor-ideas-
homebnc-v5.jpg'
```

```
FROM property WHERE title='Modern
Apartment';
```

```
INSERT INTO property_photo (property_id,
photo_url)
```

```
SELECT property_id,
'https://cloudfrontgharpediabucket.gharpedia.co
m/uploads/2022/06/Small-apartment-
Decorating-Ideas-01-0503010065-1.jpg'
```

```

FROM property WHERE title='Modern
Apartment';

INSERT INTO property_photo (property_id,
photo_url)

SELECT property_id,
'https://cdn.decorilla.com/online-decorating/wp-
content/uploads/2018/03/5ecf09d9b17b63c73fff
8d26cff80b41fc151678.jpg'

FROM property WHERE title='Family House';

INSERT INTO property_photo (property_id,
photo_url)

SELECT property_id,
'https://tse2.mm.bing.net/th/id/OIP.usdFNyvvFIT
oaw0lr37pTQHaeT?rs=1&pid=ImgDetMain&o=7
&rm=3'

FROM property WHERE title='Studio Room';

```

B.MONGO DB

```

{
  "users":[
    {
      "_id": "u1",
      "fullName": "Eden Alemayehu",
      "email": "eden@gmail.com",
      "phone": "0956748932",
      "password": "hashed_pw",
      "role": "owner",
      "createdAt": "2026-05-01"
    },
    {
      "_id": "u2",
      "fullName": "Ashenafi Gebremedhn",
      "email": "ashenafi@gmail.com",
      "phone": "0946848962",
      "password": "hashed_pw",
      "role": "tenant",
      "createdAt": "2026-05-02"
    }
  ],
  "locations": [

```

```

{
  "_id": "l1",
  "city": "Addis Ababa",
  "subcity": "Bole" },
  {
    "_id": "l2",
    "city": "Addis Ababa",
    "subcity": "Goro" }],
  "properties": [
    {
      "_id": "p1",
      "title": "2 Bedroom Apartment",
      "description": "Clean and modern
apartment",
      "propertyType": "apartment",
      "price": 15000,
      "bedrooms": 2,
      "areaSize": 120,
      "status": "available",
      "userId": "u1",
      "locationId": "l1",
      "createdAt": "2026-05-01"
    },
    {
      "_id": "p2",
      "title": "Family House",
      "description": "Spacious house with parking",
      "propertyType": "house",
      "price": 25000,
      "bedrooms": 3,
      "areaSize": 200,
      "status": "rented",
      "userId": "u1",
      "locationId": "l2",
      "createdAt": "2026-05-03"}],
  "propertyPhotos": [
    {
      "_id": "ph1",

```

```

    "photoUrl": "https://img.com/p1.jpg",
    "propertyId": "p1"
  },
  {
    "_id": "ph2",
    "photoUrl": "https://img.com/p2.jpg",
    "propertyId": "p2"
  }
]
}

```

QUERIES

```

use property_rental;
db.createCollection("users");
db.createCollection("locations");
db.createCollection("properties");
db.createCollection("propertyPhotos");
const userResult = db.users.insertMany([
  {
    fullName: "Eden Alemayehu",
    email: "eden@gmail.com",
    phone: "0956748932",
    password: "hashed_pw",
    role: "owner",
    createdAt: new Date("2026-05-01")
  },
  {
    fullName: "Ashenafi Gebremedhn",
    email: "ashenafi@gmail.com",
    phone: "0946848962",
    password: "hashed_pw",
    role: "tenant",
    createdAt: new Date("2026-05-02")
  }
]);

const u1 = userResult.insertedIds[0];
const u2 = userResult.insertedIds[1];

```

```

const locResult = db.locations.insertMany([
  {
    city: "Addis Ababa",
    subcity: "Bole"
  },
  { city: "Addis Ababa",
    subcity: "Goro"
  }
]);

const l1 = locResult.insertedIds[0];
const l2 = locResult.insertedIds[1];
const propResult = db.properties.insertMany([
  {
    title: "2 Bedroom Apartment",
    description: "Clean and modern apartment",
    propertyType: "apartment",
    price: 15000,
    bedrooms: 2,
    areaSize: 120,
    status: "available",
    userId: u1,
    locationId: l1,
    createdAt: new Date()
  },
  {
    title: "Family House",
    description: "Spacious house with parking",
    propertyType: "house",
    price: 25000,
    bedrooms: 3,
    areaSize: 200,
    status: "rented",
    userId: u1,
    locationId: l2,
  }
]);

```

```

        createdAt: new Date()
    });

    const p1 = propResult.insertedIds[0];
    const p2 = propResult.insertedIds[1];
    db.propertyPhotos.insertMany([
        { photoUrl: "https://img.com/p1.jpg", propertyId:
p1 },
        { photoUrl: "https://img.com/p2.jpg", propertyId:
p2 }
    ]);
    db.users.find();
    db.properties.find({ status: "available" });
    db.properties.find({ price: { $lt: 20000 } });
    db.properties.aggregate([
        {
            $lookup: {
                from: "locations",
                localField: "locationId",
                foreignField: "_id",
                as: "location"
            }
        }
    ]);
    db.properties.find({ bedrooms: { $gte: 2 } });
    db.locations.find();

```

C..SERVEY QUESTIONS

PROPERTY MANAGEMENT SYSTEM SURVEY QUESTIONNAIRE

Project: Database Systems Group Assignment

Date: (Current Date)

Respondent ID: _____

Role: ☐ Property Owner ☐ Tenant ☐ Property Manager

SECTION A: GENERAL INFORMATION

1. What is your age range?
☐ 18-25 ☐ 26-35 ☐ 36-45 ☐ 46-55 ☐ 55+
2. How long have you been involved in property (renting or owning)?
☐ Less than 1 year ☐ 1-3 years ☐ 4-6 years ☐ 7+ years
3. How many properties do you currently own/rent/manage?
☐ 1 ☐ 2-3 ☐ 4-5 ☐ 6+
4. How often do you use online platforms for property-related activities?
☐ Daily ☐ Weekly ☐ Monthly ☐ Rarely ☐ Never

SECTION B: CURRENT CHALLENGES (For Property Owners)

5. What methods do you currently use to list your properties?
☐ Social media (Telegram, Facebook)
☐ Real estate agents
☐ Word of mouth
☐ Online property platforms (e.g., EthioRealEstate)
☐ Newspaper/Print ads
☐ Other: _____

- ☐ No clear pricing information
 - ☐ Cannot see enough photos before visiting
 - ☐ Difficult to contact owners
 - ☐ No way to compare multiple properties
 - ☐ Scams are common
 - ☐ Other: _____
11. How many properties do you usually need to visit before finding one?
☐ 1-2 ☐ 3-5 ☐ 6-10 ☐ 10+
 12. On a scale of 1-5, how difficult is it to find reliable property information?
(1 = Very easy, 5 = Very difficult)

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

SECTION D: SYSTEM REQUIREMENTS

13. Which features would you find most valuable? (Select top 3)
☐ Search properties by location (city/subcity)
☐ Filter by price range
☐ View multiple property photos
☐ Contact owner directly from listing
☐ Property status (available/rented)
☐ Save favorite properties
☐ Compare properties side-by-side
☐ Map view of property locations
☐ Tenant rating system for owners
☐ Payments/rent tracking

6. What challenges do you face when listing properties? (Select all that apply)
☐ Difficult to find qualified tenants
☐ No proper system to track property status
☐ Hard to manage multiple property photos
☐ No way to verify tenant history
☐ Payment tracking is manual and error-prone
☐ Properties stay empty for too long
☐ Other: _____

7. How long does it typically take you to find a tenant for a property?
☐ Less than 1 week ☐ 1-4 weeks ☐ 1-3 months ☐ 3+ months
8. On a scale of 1-5, how satisfied are you with your current method?
(1 = Very dissatisfied, 5 = Very satisfied)

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

SECTION C: CURRENT CHALLENGES (For Tenants)

9. How do you currently search for rental properties?

- ☐ Social media groups
- ☐ Asking friends/family
- ☐ Walking around neighborhoods
- ☐ Real estate agents
- ☐ Online websites
- ☐ Other: _____

10. What challenges do you face when searching for properties? (Select all that apply)
☐ Listings are often fake or already rented

14. How important is each feature to you? (1 = Not important, 5 = Very important)

Feature	Rating (1-5)
Accurate property status (available/rented)	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
Multiple property photos	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
Owner contact information	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
Price information	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
Property location (city/subcity)	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
Search and filter options	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5
User reviews/ratings	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

15. Would you use a centralized property management system if available?
☐ Definitely yes ☐ Probably yes ☐ Not sure ☐ Probably no ☐ Definitely no
16. How much would you be willing to pay monthly for such a system? (if applicable)
☐ 0 ETB (should be free) ☐ 100-300 ETB ☐ 301-600 ETB ☐ 600+ ETB

SECTION E: ADDITIONAL COMMENTS

17. What is the biggest problem with property listing/renting in your area?

18. Any other suggestions or features you would like to see?

THANK YOU FOR YOUR TIME!